

Resolution: Pros/Cons and Applications

Video Lesson

Duration: 00:06:10

Slide Deck

M09S10 (<https://canvas.tamu.edu/courses/430127/files/92570682?wrap=1>) 

Lesson Transcript

In this lecture, we look at the pros and cons of resolution and applications of the resolution

Advantages and Disadvantages of Resolution

- **Advantages:** (1) Resolution is universally applicable to problems which can be described in first-order logic. (2) The theorem proving engine can be decoupled from any particular domain.
- **Disadvantage:** (1) Resolution is too inefficient to be generally applicable. (2) This is partly because resolution is purely syntactic, and it does not consider what the predicates *mean*. For this reason, developing a domain-dependent heuristic is impossible. (3) A contradiction in the axiom set may allow anything to be proved. (4) It is difficult for a human to understand proof by resolution prover.

There are both advantages and disadvantages of resolution-based theorem proving. Most of these are based on the domain independence of the whole approach.

The advantages of resolution are: resolution is universally applicable to problems that can be described in first-order logic. Another advantage is the theorem-proving engine can be decoupled from any particular domain. So, when the domain changes, we do not need to change the theorem-proving engine.

There are also disadvantages. Resolution is too inefficient to be generally applicable, and this is partly because the resolution is purely syntactic, and it does not consider what the predicates mean. For this reason, developing a domain-dependent heuristic is impossible.

As we have seen, an informed search is far more efficient than an uninformed search. The fact that we cannot use informed search in resolution is a critical limitation.

Another issue is that a contradiction in the axiom set may allow anything to be proved, and it is difficult for a human to understand the proof generated by the resolution prover.

Question Answering: Example

Example:

- 1 $\forall x \forall y \forall z ((\text{Parent}(x, z) \wedge \text{Parent}(z, y)) \rightarrow \text{Grandparent}(x, y))$
- 2 $\forall x \forall y (\text{Mother}(x, y) \rightarrow \text{Parent}(x, y))$
- 3 $\forall x \forall y (\text{Father}(x, y) \rightarrow \text{Parent}(x, y))$
- 4 $\text{Father}(\text{Zeus}, \text{Ares})$
- 5 $\text{Mother}(\text{Hera}, \text{Ares})$
- 6 $\text{Father}(\text{Ares}, \text{Harmonia})$

Question: "Who is a grandparent of Harmonia?"

- 1 $\exists x (\text{Grandparent}(x, \text{Harmonia}))$
Negated: $\neg \exists x (\text{Grandparent}(x, \text{Harmonia}))$
 $= \forall x (\neg \text{Grandparent}(x, \text{Harmonia}))$

Resolution-based theorem provers can be used not just for proving theorems but they can also be used to answer questions based on facts and rules in the knowledge base.

Let us look at this very simple example. It contains several facts. Such that Zeus is the father of Ares, and Hera is the mother of Ares. And Ares is the father of Harmonia. Given these facts and these rules, if x is a parent of z , and z is a parent of y , then x is a grandparent of y . Also, if x is a mother of y , then x is a parent of y . If x is a father of y , then x is also a parent of y .

So, using these facts and rules, can we answer this question? Who is a grandparent of Harmonia?

So, that question can be represented like this. Does there exist x such that x is a grandparent of Harmonia? But the problem is that this will only tell us whether this is true or false, but it will not explicitly tell us what this x is. Who is that example that helps us satisfy this expression?

Question Answering: Result

- Resolution on the previous example generates **False** in the end, but what that answers is the question "**Is there a grandparent of Harmonia?**". Of course the answer is **yes**, but the question is **who?**.
- The negated question in the above examples was $\neg \text{Grandparent}(x, \text{Harmonia})$. Clearly, the binding which x ultimately receives is the desired answer!
- Observation: one substitution along the way, starting from $\neg \text{Grandparent}(x, \text{Harmonia})$, the negated conclusion, is $\{x/\text{Hera}\}$, thus *Hera* must be an answer.

Exercise: use resolution to derive **False** in the example in the previous slide.

Given the facts and rules in the previous slide, when we run the resolution theorem prover, then in the end, it will generate a **False**. But this particular result will only answer this question: is there a grandparent of Harmonia? So, it is a question about existence, not about who.

If we recall the last part, the negated conclusion part, it has this form, $\neg \text{grandparent}(x, \text{Harmonia})$. In order for this to be canceled out, we need something in the form of $\text{grandparent}(\text{someone}, \text{Harmonia})$. Then to unify these two incompatible expressions, we have to replace x with someone. For example, x is substituted by Hera.

So, if we observe the theorem proving steps and we find that at some point, this x is replaced by a known person in the domain, Hera, then we can conclude that whenever this happens, this person must be the answer.

Question Answering: Result

- Resolution on the previous example generates **False** in the end, but what that answers is the question “Is there a grandparent of Harmonia?”. Of course the answer is **yes**, but the question is **who?**.
- The negated question in the above examples was $\neg \text{Grandparent}(x, \text{Harmonia})$. Clearly, the binding which x ultimately receives is the desired answer!
- Observation: one substitution along the way, starting from $\neg \text{Grandparent}(x, \text{Harmonia})$, the negated conclusion, is $\{x/\text{Hera}\}$, thus *Hera* must be an answer.

Exercise: use resolution to derive **False** in the example in the previous slide.

Of course, it is tedious to browse through all of the resulting clauses that the resolution prover generates.

So, there is a really easy trick to extract this kind of answer through algorithmic means.

Answer Extraction

We can introduce special predicates to extract the answers.

- **Answer predicate:**
 $\neg \text{Grandparent}(x, \text{Harmonia}) \vee \text{Answer}(x)$
I
- The answer predicate has these properties:
 - It does not resolve with anything, but it keeps track of variable bindings.
 - The theorem prover recognize a clause consisting only of the *Answer* predicate as **False**.
- For example, resolution on the previous example results in:
 $\text{Answer}(\text{Hera})$
as the final clause.

Here we introduce the answer predicate. So, since we know that this x will eventually contain the answer, we add this additional answer predicate to the clause.

What happens next is whenever x is replaced by the actual answer, this x will also be replaced by that answer.

And by treating this answer predicate as **False**, whenever a single predicate answer remains, then we can say that the theorem is proven, and we simply need to look up what goes in that first argument, in the answer predicate as shown here, and we can extract the answer.

First-Order Logic: Summary

- Standard forms: prenex normal form, skolemization, CNF.
- Resolution: negated conclusion, substitution, unification, factors and resolvents.
- Theorem provers: two-pointer method, various deletion strategies, various speed up strategies.
- Application of theorem provers: question answering.

In sum, for the first-order logic part and theorem-proving part, we looked at standard forms, resolution, and theorem-proving implementation based on the two-pointer method, various deletion strategies, and various speed-up strategies.

And we looked at the application of theorem provers in the form of question answering.

In the next lecture, which will be the final lecture on the topic of logic, we look at Prolog and theorem prover examples based on my own LISP-based implementation.

CSCE
625

MODULE 9

RESOLUTION: PROS/CONS AND APPLICATIONS